

**UNIVERSIDAD POLITÉCNICA DE  
SAN LUIS POTOSÍ**

**CARRERA:** Ingeniería en Tecnologías de la Información

**MATERIA:** CNO IV Seguridad Informática

**TRABAJO:** Lab File path traversal, traversal sequences stripped with  
superfluous URL-decode

**Ojeda Camacho José Carlos / 181629**

**PROFESOR:** Servando Contreras

## Contenido

|                                                |   |
|------------------------------------------------|---|
| Información del laboratorio .....              | 2 |
| Objetivo del laboratorio.....                  | 2 |
| Explicación técnica de la vulnerabilidad ..... | 2 |
| Procedimiento paso a paso .....                | 3 |
| Explicación del payload .....                  | 6 |
| Mitigación recomendada.....                    | 6 |

## Información del laboratorio

Nombre del laboratorio: Lab File path traversal, traversal sequences stripped with superfluous URL-decode

Nivel (Apprentice / Practitioner): Practitioner

Tipo de explotación: PATH TRAVERSAL

## Objetivo del laboratorio

El objetivo de este laboratorio es explotar una vulnerabilidad de path traversal en la visualización de imágenes de productos donde la aplicación bloquea secuencias de escape comunes (como ../) y realiza una decodificación URL de la entrada; para resolverlo, debes evadir esta restricción lógica mediante el uso de una doble codificación URL y lograr extraer con éxito el contenido del archivo crítico del sistema operativo, /etc/passwd.

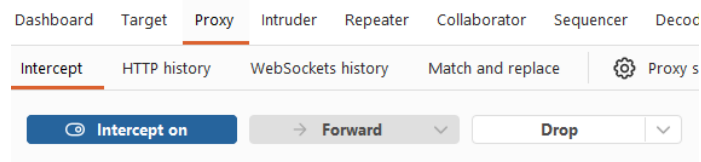
## Explicación técnica de la vulnerabilidad

La vulnerabilidad radica en una deficiencia en el control de acceso y en la validación de entradas de la aplicación, la cual intenta mitigar el riesgo de path traversal decodificando la entrada y bloqueando únicamente las secuencias relativas de navegación (como ../); sin embargo, al aplicar la validación de seguridad de manera superficial o procesar la entrada de forma inconsistente, el sistema permite que una carga útil doblemente codificada en URL eluda los filtros implementados.

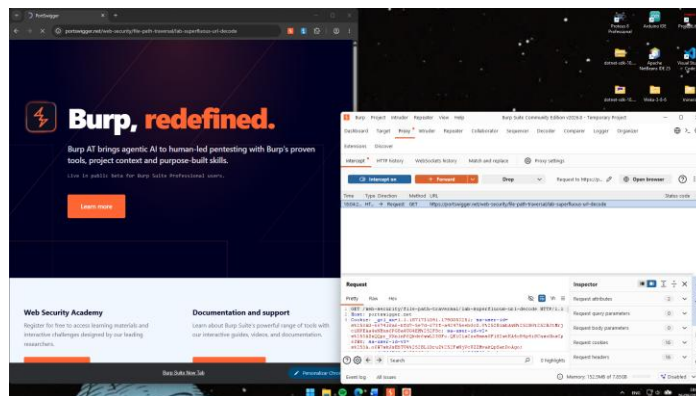
Al decodificar el payload una sola vez en la fase de inspección, no detecta el salto, pero posteriormente el backend lo procesa en su forma decodificada, lo que permite a un atacante acceder de manera ilegítima a recursos críticos del sistema operativo, tales como el archivo /etc/passwd.

# Procedimiento paso a paso

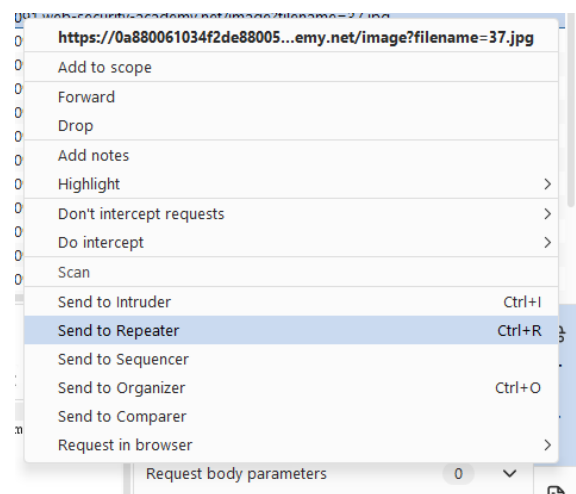
El proceso inicia con la apertura de la aplicación Burp Suite y el uso de su navegador integrado, el cual evita conflictos de configuración con certificados de seguridad. Posteriormente, en la sección **Proxy > Intercept**, se verifica que la función de intercepción se encuentre activada (*Intercept is on*). Esto permite pausar y capturar de manera temporal las peticiones HTTP generadas por el navegador antes de que alcancen el servidor de la aplicación, facilitando así su análisis y posterior modificación.



El uso del navegador integrado en Burp Suite facilita el análisis de tráfico web al enrutar las solicitudes de forma automatizada sin requerir configuraciones adicionales de certificados de seguridad. Una vez que la intercepción está activa y la petición queda retenida en el panel de control, la opción Forward permite liberar individualmente cada paquete HTTP retenido, enviándolo hacia su destino final en el servidor, o bien continuar el flujo normal de navegación una vez que se han realizado las inspecciones o modificaciones necesarias.



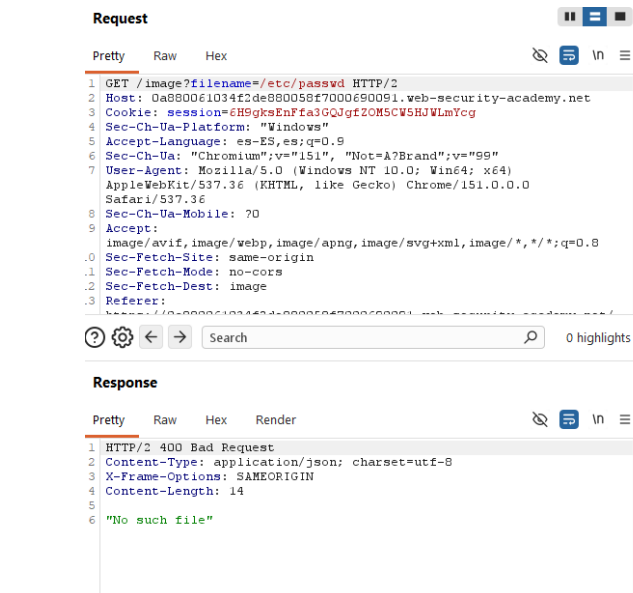
Una vez capturada la solicitud HTTP objetivo en el historial del proxy o durante la intercepción (específicamente la petición GET /image?filename=), se hace clic derecho sobre ella para desplegar el menú contextual y se selecciona la opción **Send to Repeater**. Esta acción traslada la petición a la herramienta Repeater de Burp Suite, la cual permite reenviar de forma manual y controlada tantas solicitudes como sea necesario, facilitando la prueba de iteraciones y modificaciones en los parámetros sin requerir una nueva interacción directa desde el navegador web.



Dentro de la herramienta Repeater, se procede a modificar los parámetros de la petición HTTP capturada. Específicamente, en la solicitud GET /image?filename=, se realizan los siguientes intentos para comprender el filtro de seguridad:

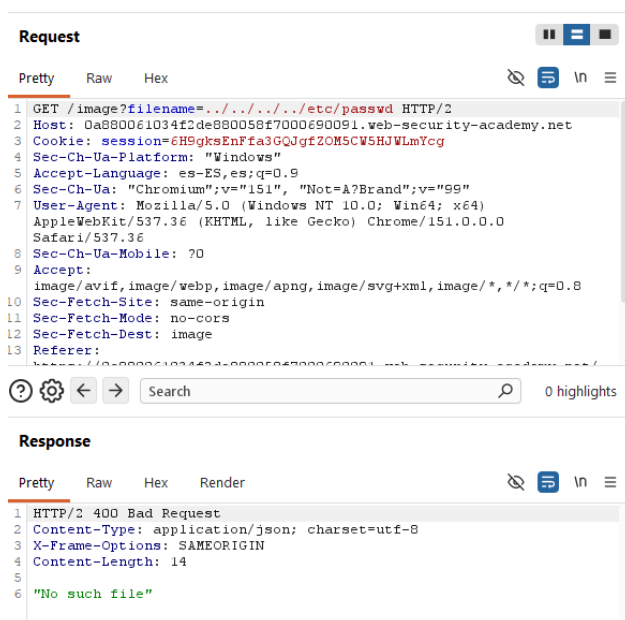
### 1. Intento 1: Ruta absoluta (/etc/passwd).

Se reemplaza el nombre de la imagen con la ruta absoluta. El servidor responde con "No such file", lo que indica que busca el archivo dentro de su directorio de trabajo predeterminado.



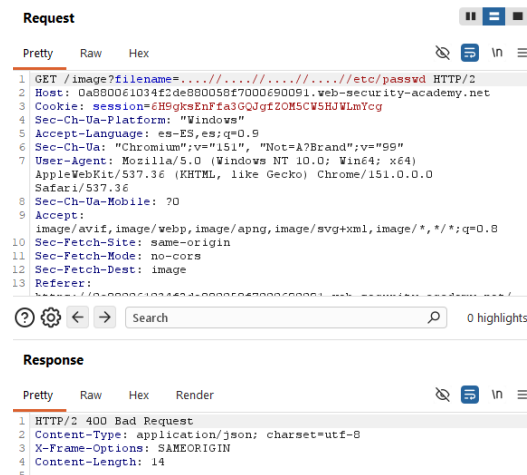
### 2. Intento 2: Salto de directorio básico (../../etc/passwd).

Se intenta un path traversal convencional. El servidor responde nuevamente con "No such file", confirmando que existe un filtro que bloquea secuencias relativas de navegación.



### 3. Intento 3: Evasión de filtro no recursivo (....//....//....//etc/passwd).

Se emplea la técnica de anidamiento de secuencias para eludir un filtro que elimine ../ una sola vez. La respuesta sigue siendo "No such file", indicando que el sistema implementa una validación más robusta.

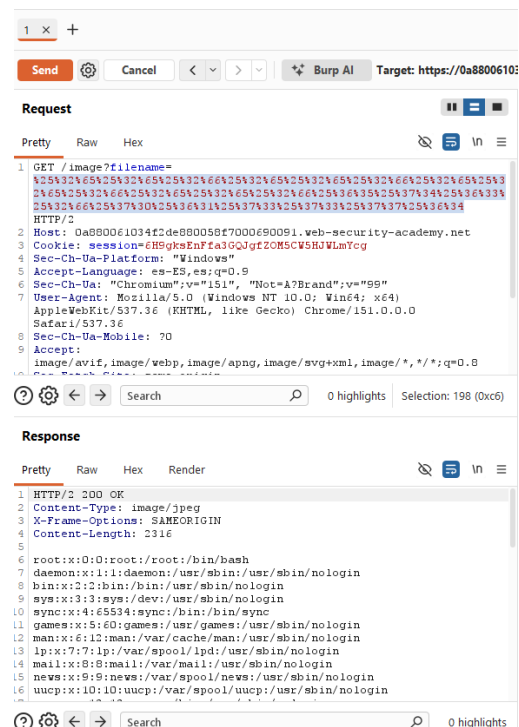


Finalmente, para evadir los controles de seguridad, se tomó la secuencia de salto básica (../...//etc/passwd) y se codificó en formato URL una primera vez. Después, este resultado se codificó una segunda vez, generando el payload final:

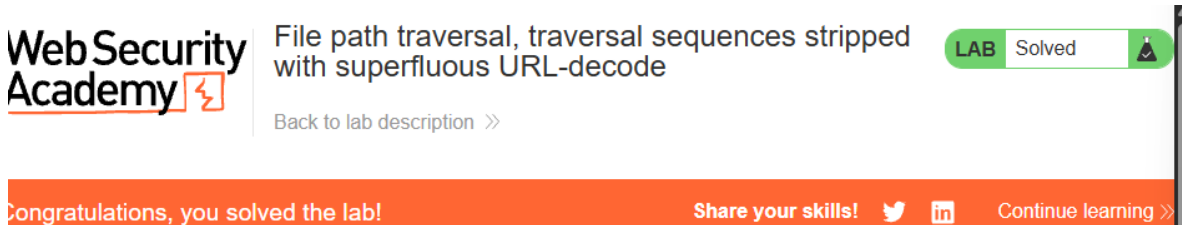
%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd. Al reemplazar el nombre del archivo de imagen original con esta cadena, el ataque funcionó con éxito.

El objetivo de esta inyección es aprovechar una deficiencia en la validación de las entradas del servidor. Dado que la aplicación bloquea las secuencias relativas de salto de directorio (como ../), la inyección de la secuencia doblemente codificada permite evadir dicho filtro, ya que la primera decodificación convierte la carga en %2e%2e%2f (evadiendo la lista negra). El archivo /etc/passwd es un estándar en pruebas de concepto (PoC) para vulnerabilidades de Path Traversal o Local File Inclusion (LFI), ya que es un archivo presente por defecto en sistemas basados en Unix/Linux que contiene información estructurada sobre las cuentas de usuario del sistema operativo.

Al enviar la solicitud modificada, se observa que el servidor la procesa correctamente y devuelve una respuesta con el código de estado HTTP/2 200 OK. En el cuerpo de la respuesta HTTP, el servidor expone el contenido íntegro del archivo /etc/passwd solicitado, mostrando claramente los registros de usuarios del sistema.



Esto confirma la explotación exitosa de la vulnerabilidad, demostrando que un atacante puede acceder a archivos sensibles alojados en el servidor.



## Explicación del payload

El payload utilizado en este laboratorio consiste en una ruta de navegación relativa codificada doblemente en formato URL, específicamente:

`%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd` Esta estructura se introduce en el parámetro afectado y elude el filtro de la aplicación —el cual está diseñado únicamente para detectar y bloquear caracteres de salto de directorio literales—, ya que al decodificar `%25` se obtiene un `%`, dejando el payload como `%2e%2e%2f`. Al no encontrar coincidencias exactas con la secuencia bloqueada, el filtro lo deja pasar, permitiendo que la función de lectura de archivos del servidor interprete la entrada en su forma final y revelando así la información confidencial solicitada.

## Mitigación recomendada

Para prevenir vulnerabilidades de Path Traversal como la analizada en este laboratorio, es necesario implementar un control riguroso sobre la validación y el procesamiento de las entradas del usuario. Las principales medidas de mitigación incluyen:

- **Validación posterior a la decodificación:** Asegurarse de que cualquier filtro de seguridad o verificación de listas negras se aplique *después* de que la entrada del usuario haya sido decodificada completamente (URL, Base64, etc.) hasta su representación final o canónica, evitando evasiones por doble codificación.
- **Uso de funciones de mapeo seguro (Basename):** Utilizar funciones integradas en el lenguaje de programación (como `basename()` en PHP o equivalentes) que extraigan únicamente el nombre del archivo solicitado, descartando cualquier ruta o directorio previo proporcionado en el parámetro.
- **Validación mediante listas blancas (Whitelisting):** Restringir los valores permitidos en los parámetros de entrada a un conjunto estricto de nombres de archivos válidos o identificadores predefinidos, rechazando cualquier cadena que contenga caracteres especiales, barras o rutas relativas/absolutas.